2-Wide RV32I Out-of-Order Core

RTL Microarchitecture Design Specification

Workload-Proportional Execution Resources, Dynamic Scheduling, In-Order Retirement, and Precise Recovery

Architecture Frozen: Hazard-Complete RTL Specification with Future Upgrade Roadmap

1 Executive Architecture Summary

This document specifies a 32-bit RISC-V (RV32I), integer-only, single-core out-of-order superscalar processor. The machine fetches, decodes, renames, dispatches, issues, writes back, and commits up to two instructions per cycle. Instructions execute out of program order and always commit in program order.

The design follows a BOOM-inspired organization: a register alias table (RAT) performs renaming onto a 60-entry physical register file, a 28-entry reorder buffer (ROB) provides precise in-order retirement, and a unified 12-entry reservation station pool performs dynamic scheduling. Execution is carried by three ports whose sizing is proportional to real RV32I instruction mix rather than a symmetric one-class-per-port layout: two ALU-capable resources, one of which additionally resolves branch and JALR outcomes, and a single shared address-generation path serving both loads and stores. Branch handling is static (not-taken, corrected explicitly at commit); no dynamic predictor, branch history table, or speculative memory disambiguation is included, since none is justified at this design point. Memory ordering is enforced by a dedicated store queue that tracks in-flight store addresses and data individually, rather than a single conservative in-flight flag. Recovery from a misprediction or exception is checkpoint-based: a full snapshot of rename state is restored in one cycle, paired with a parallel, non-walking squash of younger in-flight instructions.

2 Design Goals and Resource Allocation

Every sizing decision in this document answers one question: does this resource match how often it is actually used by representative RV32I integer code. Table 1 gives the workload composition that drives every port and structure decision that follows.

Class	Share	Sizing implication
ALU (arith/logic/imm)	40–50%	needs near two ports of throughput
Loads	20–25%	share one AGU/LSU port with stores
Stores	8–12%	address/data captured, memory write deferred to commit
Branch/JAL/JALR	15–20%	does not justify a dedicated port

Table 1: Representative RV32I integer instruction mix.

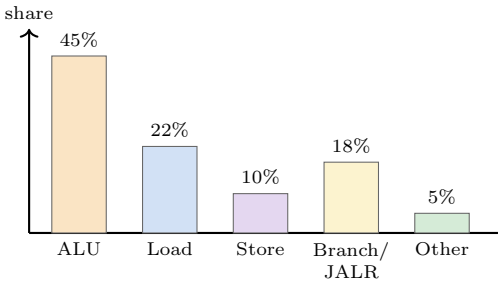

Figure 1: Representative dynamic instruction mix used for resource sizing. ALU work dominates at roughly 45%; loads and stores together account for close to a third of the stream; branches and JALR sit under a fifth.

Two consequences follow directly. First, ALU work alone is close to saturating two execution ports, so a second full ALU-class port is required, not optional. Second, branches at 15 to 20 percent of the stream cannot justify an entire dedicated port that would sit idle roughly four cycles out of five; branch resolution is folded onto the second ALU port instead. A naively symmetric one-port-per-class layout would waste issue bandwidth on the branch port and starve ALU throughput; Table 2 shows the chosen allocation instead.

Class	Share	Resource allocated
ALU-class	∼65%	two full ports, EXU0 and EXU1
Load/store	∼32%	one shared AGU/LSU port
Branch/JALR	∼18%	shares EXU1, no dedicated port

Table 2: Execution resource allocation tracks workload share rather than a symmetric one-to-one-to-one split.

Sharing EXU1 between ALU and branch work means that port performs real arithmetic work roughly 80% of the time and branch resolution the remaining 20%, instead of sitting idle 80% of the time waiting for the next branch: a direct, quantitative consequence of the instruction mix, not an arbitrary simplification.

A branch history table exists in larger designs to reduce wrong-path fetch before a misprediction is caught. Here, misprediction recovery is already a fixed, small, constant-latency operation regardless of how many branches are outstanding, achieved through checkpointing (Section 16). Static not-taken is intentionally chosen to eliminate BTB/BHT state and fetch-stage predictor timing and area overhead; its accuracy is poor specifically on taken, backward, loop-closing branches (for example, a tight loop that iterates many times before exiting is mispredicted on every iteration but the last), and the de-

sign accepts that performance cost as a deliberate minimality tradeoff rather than a claim that static prediction is broadly accurate. Because recovery cost does not scale with speculation depth, a dynamic predictor buys throughput at a complexity cost not justified at this design point (revisited in Section 22).

The remaining structures, the 28-entry ROB, the 12-entry unified reservation station pool, the 14-entry store queue, and the 6-entry checkpoint buffer, are each sized to the resource that actually bounds them, detailed alongside their own sections rather than repeated here.

3 Architectural Parameters

Table 3 fixes the single configuration used throughout the remainder of this document.

Parameter	Value	Notes
XLEN	32	RV32I word width
Fetch/Decode/ Rename/Dispatch	2	sustained every stage
Issue width	2	across 3 ports
Writeback width	2	two broadcast slots
Commit width	2	in order, gated pair
Architectural registers	32	x0 to x31
ROB depth	28	in-flight window
Physical registers	60	32 + ROB depth
Reservation stations	12	unified pool
Store queue depth	14	one shared AGU/LSU port
Checkpoint buffer	6	in-flight branches
Execution ports	3	EXU0, EXU1, AGU/LSU

Table 3: Final architectural parameters. This is the configuration used throughout the document.

Why these sizes. The ROB depth of 28 is not sized for an idealized 2 IPC target: with one ALU-class port effectively saturated by the 40 to 50 percent ALU share, and the single AGU/LSU port serialized across loads and stores, sustained throughput realistically sits near 1.3 to 1.5 IPC. A deeper window buys little beyond this point and only adds comparator and CAM cost across the ROB, reservation stations, and store queue. The physical register count follows directly: 32 architectural plus 28 speculative, exactly the ROB depth, since that is the maximum number of destinations that can ever be in flight simultaneously. The reservation station pool is roughly ROB depth over two: two real execution-capable ALU ports rarely need a deeper ready-instruction backlog than that. The store queue is sized to the load/store unit's real issue rate, one shared port, not to reservation station depth. The checkpoint buffer is small because static prediction means there is no deep speculation to exploit; six checkpoints comfortably exceeds the number of unresolved branches a 2-wide frontend feeding this ROB will realistically hold.

4 Whole Processor Architecture

Figure 2 is the single most important diagram in this document: every subsequent section describes one block or one connection shown here. Four structural groups matter: the frontend (blue) renames and allocates two instructions per cycle; the in-flight state group (green) holds all speculative bookkeeping; execution, writeback, and memory (orange/purple) do the actual work; and commit and recovery (red) are the sole authority for both architectural state changes and for undoing speculation.

Instructions enter through fetch and move as a 2-wide group through decode, rename, and dispatch, at which point they scatter into the ROB, the reservation stations, and, for stores, the store queue. From there execution is fully dynamic: an instruction issues the cycle its operands become ready and a compatible port is free, independent of the order in which it was fetched. Results converge on a two-slot writeback bus that simultaneously updates the physical register file, wakes waiting reservation-station entries, and marks ROB entries complete. Commit alone walks the ROB in strict program order, retiring up to two instructions per cycle, and is the only stage permitted to make architectural state irreversible or to trigger recovery.

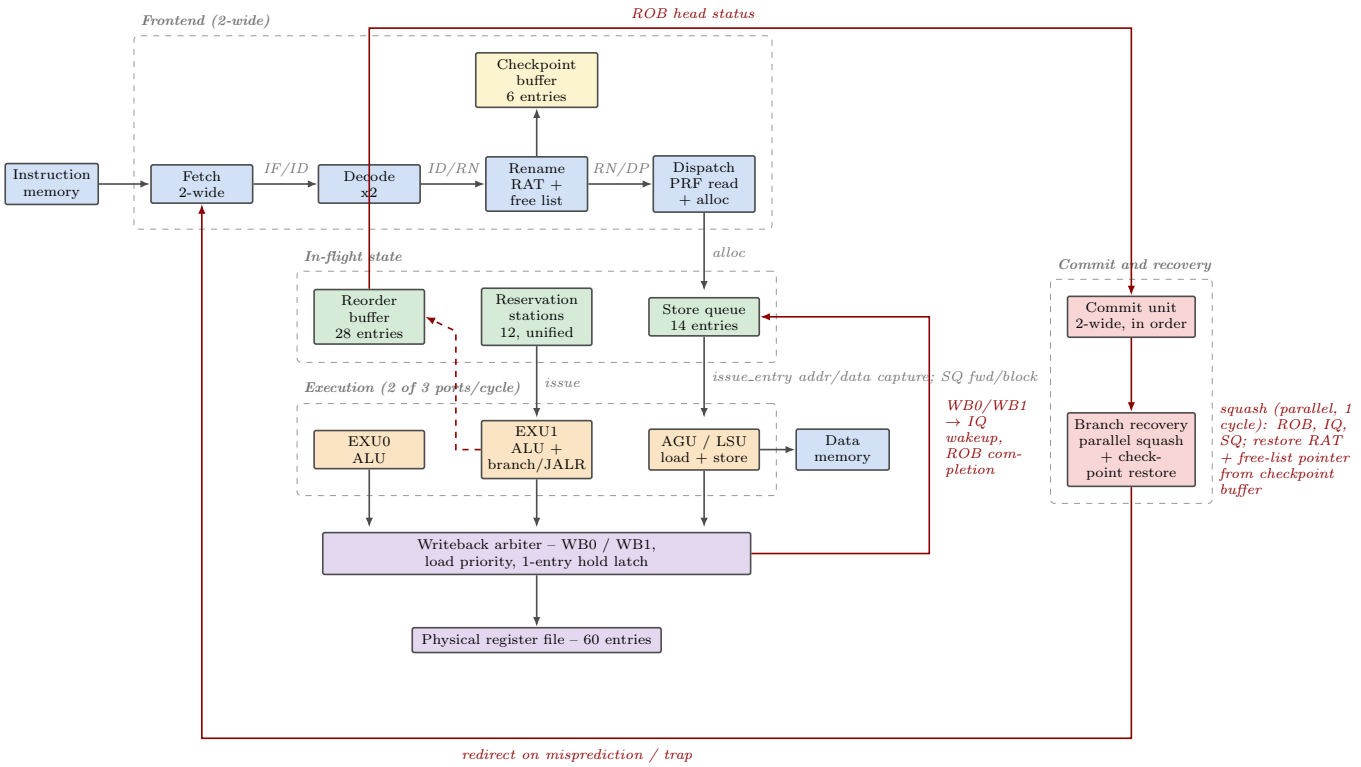

Figure 2: Top-level microarchitecture. Blue: frontend and memory, 2-wide throughout. Green: speculative in-flight state. Orange: three execution ports sized to instruction mix. Purple: writeback and register file. Red: in-order commit and checkpoint-based recovery. Grouped arrows stand in for the full one-to-many fan-out inside each dashed region.

5 Pipeline and Instruction Life Cycle

The frontend uses explicit staged pipeline registers (IF/ID, ID/RN, RN/DP); the backend, from dispatch onward, is a dynamically scheduled window with no fixed per-instruction stage count. Figure 3 gives the logical skeleton every instruction passes through; only execute and writeback behavior differs by class.

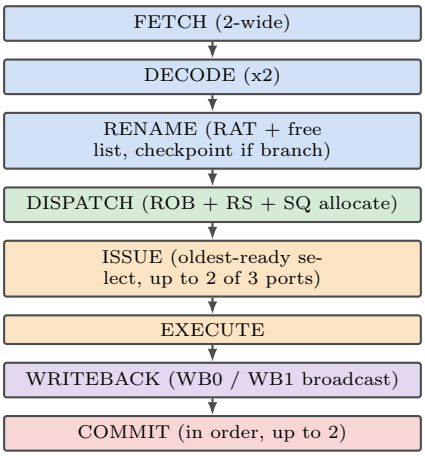

Figure 3: Pipeline skeleton common to all instruction classes.

Worked example: two independent ALU instructions

- **Fetch, IF/ID:** both instructions fetched together in slots A and B.
- **Decode, ID/RN:** both decode independently to optype ALU.
- **Rename, RN/DP:** each receives a fresh physical destination from the free list; no cross-slot dependency, so no bypass is needed.
- **Dispatch:** operands resolved; one ROB entry and one reservation-station entry allocated per slot, same cycle.
- **Wait/wakeup:** if both operands are already available, both proceed to issue almost immediately.
- **Issue:** the two-wide select can pick both the same cycle, one to EXU0 and one to EXU1.
- **Execute:** both ALUs compute results the same cycle.
- **Writeback:** both results placed on WB0 and WB1 the same cycle, no arbitration conflict.
- **Commit:** once both reach the ROB head with nothing older outstanding, both retire together.

JAL keeps a decode-time special path: zero register operands, target known at decode, redirects fetch immediately, and allocates a ROB entry without ever entering the reservation stations.

6 Frontend

6.1 Fetch

The only frontend state is the fetch program counter. Slot A reads at the current value; slot B reads four bytes ahead. Next-value selection is a fixed-priority mux.

Class	Execute / writeback
ALU	EXU0 or EXU1, single cycle, broadcasts result
Branch/ JALR	EXU1 only; resolves target and mispredict bit; broadcasts to ROB only
Load	AGU/LSU; checks store queue for forward/block before memory access
Store	AGU/LSU; address and data captured into store queue entry; no broadcast, memory write deferred to commit

Table 4: Per-class execute and writeback divergence.

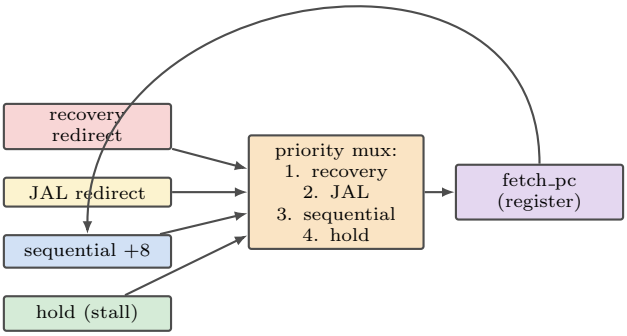

Figure 4: Fetch program-counter next-value mux. Recovery redirect always wins; only one recovery source can be active at a time, so a branch misprediction target and an exception trap vector share a single redirect input.

No branch history table exists; every branch and JALR is treated as a not-taken guess, corrected explicitly on resolution.

6.2 Pipeline registers

Three two-slot pipeline registers, IF/ID, ID/RN, and RN/DP, carry instructions through the frontend. Each uses the same two disciplines throughout the design: an enable that freezes contents under backpressure (`dispatch_stall`), and a clear that synchronously drops valid bits during recovery (`global_kill`).

6.3 Decode

Purely combinational, run twice per cycle. Produces, per slot: operation type, ALU control code, register-use flags, immediate value, and function code, mapped directly into the uop structure consumed by rename and dispatch.

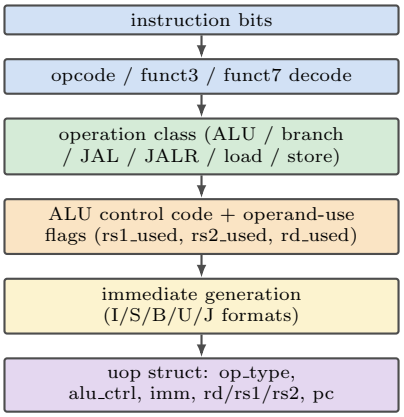

Figure 5: Decode / control-unit structure. Purely combinational; every field maps directly into the uop structure carried forward by rename and dispatch.

Op. type	rs1	rs2	rd
ALU register	yes	yes	yes
ALU immediate	yes	no	yes
Branch	yes	yes	no
JAL	no	no	yes, at dispatch
JALR	yes	no	yes
Load	yes	no	yes
Store	yes	yes (data)	no

Table 5: Operand-use flags produced by decode, per operation class.

JAL is detected at decode: its target is computed directly from the fetched PC and immediate, and it redirects fetch the same cycle without waiting for rename or dispatch. When slot A decodes to JAL, slot B is invalidated for that cycle, a purely local truncation of the fetch group.

6.4 Frontend stall and flush

`dispatch_stall` freezes the fetch PC and all three frontend pipeline registers whenever the ROB, reservation stations, store queue, free list, or checkpoint buffer lack space for the incoming group. `global_kill` synchronously clears valid bits in every frontend register during recovery, detailed fully in Section 16 and Section 19.

7 Register Renaming

Renaming eliminates write-after-read (WAR) and write-after-write (WAW) hazards structurally: every destination-writing instruction receives a fresh physical register, so no instruction can ever be forced to wait for, or accidentally overwrite, a value another instruction still needs. The rename unit additionally owns checkpoint allocation for branches (Section 7.5).

7.1 Register Alias Table

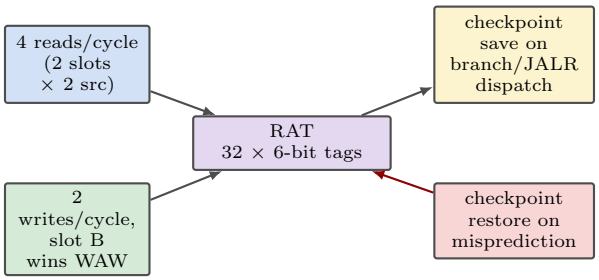

Figure 6: RAT read/write/checkpoint datapath. Writes are gated by a destination-valid flag; slot B's write overrides slot A's on a same-cycle same-register conflict.

pdst, pdst_old, has_dest. Every renamed instruction carries `has_dest` (does it write an architectural register), `pdst` (its freshly allocated physical tag if so), and `pdst_old` (the physical tag it displaces in the RAT, recorded so the ROB can push it back to the free list at commit).

7.2 Free List

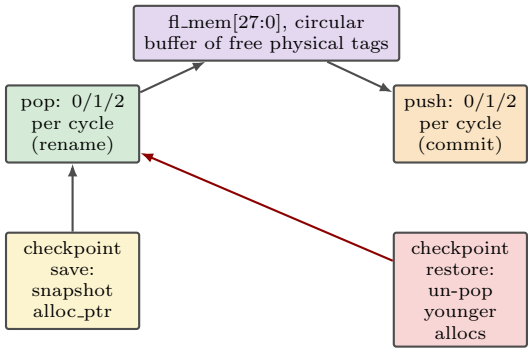

Figure 7: Free list, 28 entries, sized to ROB depth since at most that many destinations can be in flight at once. Independent `alloc_ptr`/`reclaim_ptr`; checkpointing the allocation pointer alone is sufficient to undo any sequence of speculative allocations.

Restoring `alloc_ptr` to its checkpointed value discards every allocation made after the checkpoint without disturbing any reclamation, since reclamations only ever come from in-order commits of instructions that are, by construction, never squashed.

Next-state. `alloc_ptr` and `reclaim_ptr` are independent counters, each advancing by the actual number of allocations or reclamations that cycle (0, 1, or 2), so a same-cycle allocation and reclamation update both pointers without interaction:

$$\text{alloc\_ptr}' = \text{alloc\_ptr} + n_{\text{pop}}, \qquad \text{reclaim\_ptr}' = \text{reclaim\_ptr} + n_{\text{push}}$$

$$\text{fl\_count}' = \text{fl\_count} - n_{\text{pop}} + n_{\text{push}}, \qquad n_{\text{pop}}, n_{\text{push}} \in \{0, 1, 2\}$$

On a recovery restore this cycle, `alloc_ptr` instead takes the checkpointed value directly (any same-cycle rename pop is already suppressed by `global_kill`, since dispatch and rename are frozen during recovery); `reclaim_ptr` and `fl_count` are unaffected, since a restore never invalidates a push. At reset, `alloc_ptr` = `reclaim_ptr` = 0 and `fl_mem` holds physical tags 32 through 59 in order, giving `fl_count` = 28.

7.3 Same-Group Dependency

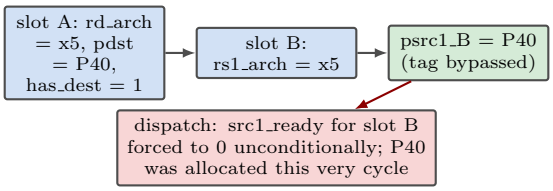

Figure 8: Same-group rename bypass. The tag is bypassed directly from the sibling slot's fresh destination; readiness is never bypassed this way.

If slot A writes x5 and slot B reads x5 in the same rename group, slot B's source tag is bypassed directly from slot A's freshly allocated destination tag rather than read from the RAT, which would still hold the stale mapping.

7.4 Same-Cycle WAW

If both slots write x5, only slot B's RAT write survives the cycle, which falls out naturally from ordering slot B's write after slot A's within the same clocked write, requiring no dedicated hazard check. The physical register slot A had claimed is not wasted: it becomes the `pdst_old` recorded in slot A's own ROB entry and is reclaimed once slot A itself retires.

7.5 Checkpoint State

Every branch or JALR takes a full snapshot of the RAT and the free-list allocation pointer at the moment it renames, stored in one of 6 checkpoint buffer slots alongside the branch's own ROB index. A misprediction restores both in a single cycle rather than unwinding state instruction by instruction. If the checkpoint buffer is full, dispatch of a group containing a branch stalls (`checkpoint_full_stall`); this bounds the machine to 6 unresolved branches in flight, comfortably exceeding what a 28-entry ROB with a single branch-capable port will realistically hold. A checkpoint is released the instant its branch resolves correctly, well before that branch itself commits.

Recovery precedence and younger checkpoints. New checkpoint allocation and normal release both happen only at dispatch and at branch resolution respectively, both of which are suppressed by `global_kill` during recovery, so a checkpoint can never be allocated or normally released in the same cycle recovery is restoring one; no priority rule between them is needed. Every checkpoint entry still associated with a branch strictly younger than the recovery point is squashed by the same parallel age comparator built for the ROB, reservation stations, and store queue (Section 10), using that checkpoint's own stored ROB index, and is returned to the free checkpoint pool the same cycle, since a squashed branch can never resolve to release it normally.

8 Physical Register File

60 entries (32 architectural baseline plus 28 speculative), each a 32-bit value plus a ready bit. Four read ports serve dispatch (2 slots by 2 sources), two independent write ports serve writeback (WB0/WB1), two dispatch-time write ports allow both dispatch slots to be a JAL in the same cycle, and two clear-ready ports serve rename allocation.

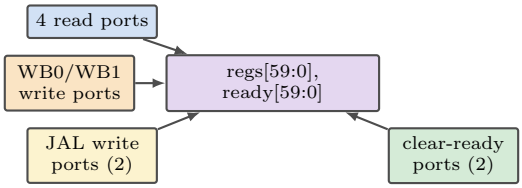

Figure 9: Physical register file organization. Six write-capable ports never target the same address in the same cycle: each port's address source is disjoint by construction, an in-flight producer's own tag versus a tag freshly allocated this very cycle.

Tag, value, and ready state are kept strictly separate throughout this design. The physical tag is a fixed identity assigned at rename and never changes. The value is meaningless until the producer writes back. Readiness is the single bit answering whether the value can currently be trusted, tracked independently of both. Architectural register x0 is never renamed and never allocated a physical destination; `has_dest` is forced to 0 end to end for any instruction with `rd_arch` equal to zero. At reset, `ready[i]` is 1 for the 32 architectural-baseline tags (0 through 31, holding the architectural reset value of their register) and 0 for the 28 speculative tags (32 through 59), which are not yet holding any committed value.

9 Dispatch and Operand Resolution

Dispatch is the bridge between the frontend, which only knows physical tags, and the dynamic scheduler, which needs to know whether each tag's value is already available. It is purely combinational. Each source operand is resolved through a three-level priority, checked independently per slot per source.

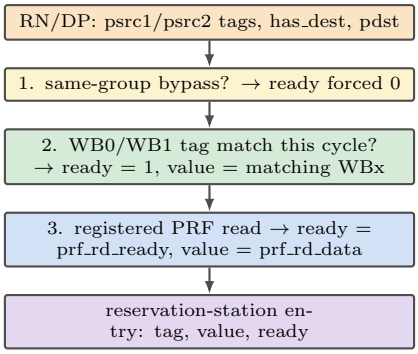

Figure 10: Operand-resolution priority. Level one is the same-group bypass override; level two checks both writeback ports independently; level three falls back to the registered physical register file state.

Two writebacks can never target the same physical tag in the same cycle, since a tag has exactly one producer in flight at a time by construction of renaming; level two therefore never needs a priority rule between its two comparisons. Dispatch allocates, per cycle: up to two ROB entries, up to two reservation-station entries, and, for stores, up to two store-queue entries. All allocation is suppressed during recovery. A dispatch group either allocates completely across every resource it needs or stalls completely; there is no partial allocation across slots, since a group that half-allocates leaves inconsistent state across structures that must stay in lockstep.

10 Reorder Buffer

Execution may be out of order; the ROB preserves program order. It is a 28-entry circular buffer with two-wide allocation and two-wide commit, holding the minimum information needed for retirement and nothing more: store data lives in the store queue, and RAT/free-list snapshots live in the checkpoint buffer, each referenced from the ROB entry by pointer only.

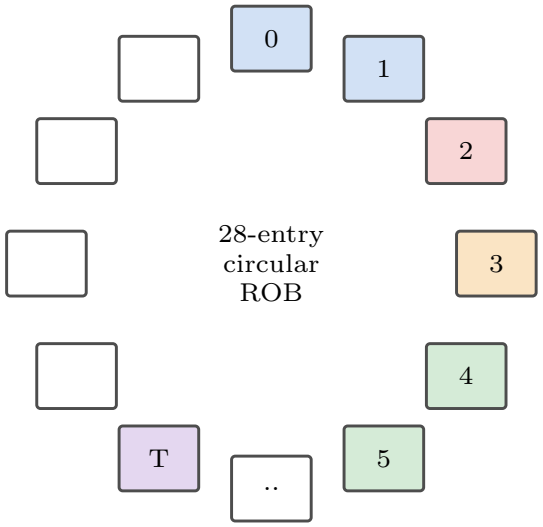

Figure 11: ROB as a circular structure (12 of 28 slots shown). Commit reads two head entries every cycle; allocation writes at the tail. Recovery invalidates every entry younger than a recovery point in one cycle via parallel age comparators rather than a sequential walk.

rob_entry fields
valid, done, pc
exception, exception_cause
op_type
rd_valid, arch_rd, new_pdst, old_pdst
is_branch, mispredicted, redirect_target, ckpt_id
is_store, sq_idx

Figure 12: Minimal ROB entry. Store and checkpoint fields are single pointers, never duplicated payload.

Structure	Owns
ROB	program order, completion, exception status, destination bookkeeping, pointers into SQ and checkpoint buffer
Reservation stations	everything needed to schedule/execute: tags, values, readiness, control fields; discarded the instant an entry issues
PRF	actual result values, never duplicated into the ROB
Store queue	store address and data; the ROB entry holds only `sq_idx`
Checkpoint buffer	RAT and free-list snapshots; the ROB entry holds only `ckpt_id`

Table 6: Ownership boundaries. No information is duplicated across structures.

Allocation writes at tail (and tail+1), gated by available space; completion sets the done bit for any entry targeted by WB0 or WB1; commit clears valid at head (and head+1) when both eligibility conditions hold. Allocation, completion, and commit can all occur the same cycle without conflict, since allocation writes at the tail

while commit clears at the head, ranges that cannot overlap while the buffer is neither empty nor full.

Next-state. `head` and `tail` are ROB_IDX_W-bit counters that advance by the actual number of commits or allocations that cycle (0, 1, or 2), wrapping via ordinary modular arithmetic; `count` is the sole full/empty discriminator:

$$\text{tail}' = \text{tail} + n_{\text{alloc}}, \qquad \text{head}' = \text{head} + n_{\text{commit}}$$

$$\text{count}' = \text{count} + n_{\text{alloc}} - n_{\text{commit}}, \qquad n_{\text{alloc}}, n_{\text{commit}} \in \{0, 1, 2\}$$

On a recovery cycle, allocation is suppressed ($n_{\texttt{alloc}} = 0$ under `global_kill`) and `tail` instead takes `branch_idx+1` directly, the index one past the mispredicted branch's own entry; `count` is recomputed from the new `tail` and unchanged `head` rather than accumulated, since the squash removes a number of entries not otherwise tracked by a simple counter step. At reset, `head` = `tail` = 0, `count` = 0, and every entry's `valid` bit is 0.

$$\text{age}(i) = (\text{idx}_i - \text{head}) \bmod \text{ROB\_DEPTH}$$

$$\text{squash}(i) = \text{valid}_i \wedge \big(\text{age}(i) > \text{age}_{\text{branch}}\big)$$

This comparator is replicated once per ROB entry, once per reservation-station entry, once per store-queue entry, and once per checkpoint-buffer entry (Section 7.5), so the entire squash resolves combinationally in a single cycle regardless of how many entries are discarded.

11 Reservation Stations / Issue Queue

A single unified 12-entry pool serves all three execution ports, rather than per-port stations. Unifying avoids partitioning issue bandwidth into per-class queues that would each need independent occupancy tracking, and keeps the pool from fragmenting: with only two ALU-class ports and one memory port, fixed per-class pools would routinely leave one pool empty while another backs up.

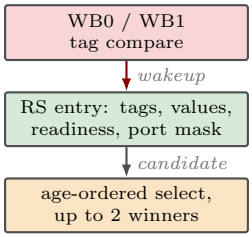

Figure 13: Reservation-station entry lifecycle: wakeup then select. Every entry compares each not-yet-ready operand against both writeback ports independently, every cycle.

Wakeup. Every not-yet-ready operand compares its tag against both WB0 and WB1 independently each cycle; either match sets ready. Two writebacks can never carry the same tag the same cycle, since a physical tag has exactly one producer in flight at a time, so no priority rule between the two ports is needed here. Comparisons are gated on `has_dest`: a plain branch, a store, or an x0-destination instruction's don't-care tag can never spuriously wake a waiting source.

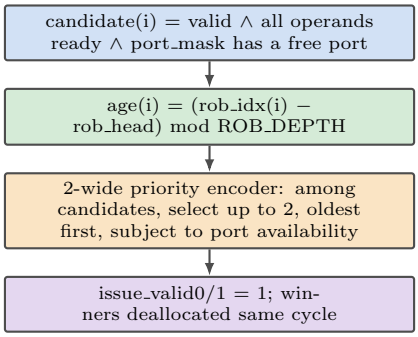

Figure 14: Select logic. A candidate needs every operand ready and a compatible free port. Among candidates, select is oldest-first by ROB-index distance from the head, feeding a 2-wide priority encoder.

Readiness determines candidacy; age determines ordering among ready candidates. Selection strictly favors readiness over position: a younger ready instruction issues ahead of an older not-yet-ready one whenever the older one is not itself a candidate this cycle. Age only breaks ties among instructions that are already ready; it never gates candidacy. This is the defining out-of-order property of the machine, and program order remains entirely the ROB's responsibility.

Issue versus recovery. Select is gated by `global_kill`: no entry is chosen and no winner deallocates on a recovery cycle, since any candidate that cycle may itself belong to the set being squashed. Normal select resumes the cycle after recovery completes, evaluated fresh against whatever entries survived the squash.

12 Execution Units and Port Arbitration

Three ports realize exactly the combinations the workload mix calls for: two ALU operations together, an ALU operation alongside a load or store, or an ALU operation alongside a branch. Up to two winners issue per cycle regardless of how many of the three ports are simultaneously free.

Port	Handles
EXU0	ALU only
EXU1	ALU, shared with branch/JALR resolution
AGU/LSU	address generation, load, store

Table 7: Execution port responsibilities.

Sharing EXU1 between ALU and branch work is the central efficiency decision in this design: a dedicated branch port would sit idle roughly 80% of cycles given a 15 to 20% branch share, while ALU demand alone approaches two ports of throughput. The AGU/LSU port computes `src1 + imm` for both loads and stores; a load, at issue, checks the store queue for older entries with unknown or matching addresses before it is allowed to proceed (Section 14); only one load or store can issue per cycle, a genuine structural hazard (Section 17).

Branch, JALR, and JAL routing. These three control-flow classes are handled by three distinct paths, not one:

- **Conditional branch**: issues to EXU1 only; produces taken/target/mispredicted metadata written directly to its own ROB entry, consumed by commit and, on misprediction, by recovery. `has_dest` is false; there is no PRF destination and nothing is broadcast on WB0/WB1.
- **JALR**: issues to EXU1 only; produces the same taken/target/mispredicted ROB metadata as a branch, and, if `rd_arch` is nonzero, additionally produces a link value (pc+4) that is broadcast on WB0/WB1 like any ordinary ALU-class result, written to the PRF and waking any waiting reservation-station entry through the normal writeback network (Section 13).
- **JAL**: never enters the reservation stations at all. Its target is computed at decode and redirects fetch that same cycle (Section 6.3); its link value, if `rd_arch` is nonzero, is written directly to its freshly allocated physical register at dispatch through the dedicated JAL write ports (Section 8), not through WB0/WB1, since the value (pc+4) is already known before dispatch and needs no execution.

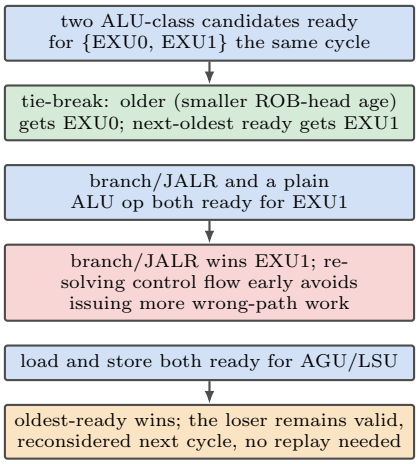

Figure 15: Deterministic port-arbitration tie-breaks. Each scenario has exactly one hardware rule; none is left ambiguous.

13 Writeback Network

Every execution result funnels onto one of two broadcast slots; the writeback arbiter is the single point where this happens.

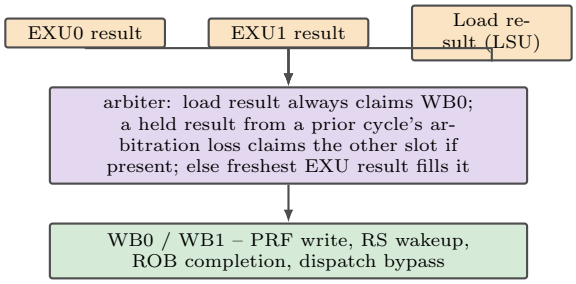

Figure 16: Writeback arbitration onto two broadcast slots.

Why a single hold latch is provably sufficient. Maximum simultaneous demand in a cycle is bounded: at most 2 fresh ALU-class results, since issue width is 2, plus at most 1 completing load, whose multi-cycle latency decouples its completion from that cycle's issue. Three demands against two slots means at most one result ever needs to be held.

Hold-latch clear. `held_valid` is cleared unconditionally by `global_kill`. Because recovery is detected at commit and the ROB retires strictly in order, nothing older than the mispredicted branch can still be producing a result at that point; any value sitting in the hold latch at the moment of recovery therefore belongs to the branch itself or to a younger, squashed instruction, so an unconditional clear is exact, not an approximation. Capture into the hold latch and consumption out of it are mutually exclusive within a cycle: the arbiter always drains `held_valid` into a slot first, so a result is never held for more than one cycle under the bounded-demand argument above.

Consumer	Action
PRF	two independent write ports, each gated by that port's destination-valid flag
Reservation stations	every entry's comparators check both ports independently
ROB	up to two entries marked done per cycle
Dispatch	same-cycle bypass, operand-resolution priority level two

Table 8: Writeback fanout: every consumer of WB0/WB1.

14 Load/Store Unit and Store Queue

A 14-entry store queue holds address and data for every in-flight store individually, rather than relying on a single conservative flag that would block every load whenever any store is in flight anywhere.

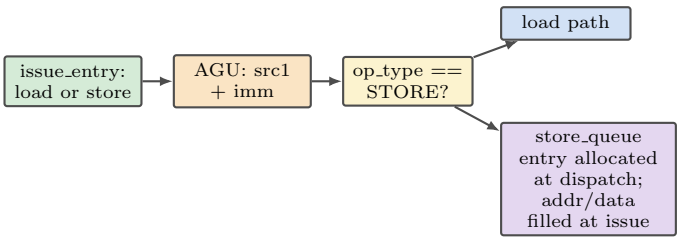

Figure 17: AGU/LSU overview. One address-generation unit serves both loads and stores; a store's address and data are captured into its own store-queue entry.

Every store-queue entry already carries the byte size of its access (1, 2, or 4 bytes, from SB/SH/SW), the same size the commit-time architectural memory write already needs; ordering uses that existing field rather than adding any new state. Two accesses overlap when $\max(\text{store\_addr}, \text{load\_addr}) < \min(\text{store\_addr} + \text{store\_size}, \text{load\_addr} + \text{load\_size})$. This is byte-range ordering, not starting-address equality: a halfword store to address $A{+}2$ and a word load from address A share bytes and must be treated as overlapping even though neither starting address matches the other. Forwarding itself remains exact-range only, the same single mechanism the design has always had; a load whose range only partially overlaps an older store's range does not incorrectly proceed to memory, but it also is not incorrectly forwarded a partial or merged value the current hardware has no path to construct, so it conservatively stalls instead. A store's address and data are speculative compute from issue until commit; the only point in the machine that modifies

Condition (checked against every older SQ entry, age-ordered)	Action
any older entry, address not yet known	load is not a candidate this cycle, regardless of any other entry's status
youngest older entry with known address whose byte range [addr, addr+size) overlaps the load's byte range, ranges identical	forward directly from that store-queue entry, no memory access
youngest older entry with known address whose byte range overlaps the load's byte range, ranges not identical	load is not a candidate this cycle; re-evaluated every cycle until that entry retires
no older entry's byte range overlaps the load's byte range	proceed to memory normally

Table 9: Load ordering policy: byte-range overlap, checked at issue against store-queue entries older than the load by ROB index. Exact-address forwarding is the identical-range case of this same rule, not a separate mechanism.

architectural memory is commit. A squashed store can never have touched memory, since its store-queue entry is invalidated by recovery before it could ever reach the ROB head.

In-flight load lifecycle. A load's AGU/LSU in-flight tracking entry, the same state referenced in Table 21, moves through a small lifecycle: issue allocates it and records the load's ROB index; it remains in-flight while the memory response is outstanding; if `global_kill` asserts while it is in-flight and its stored ROB index is younger than the recovery point, the entry is killed that same cycle, using the identical age comparison already applied to the ROB, reservation stations, and store queue (Section 16); otherwise it simply waits for the response. A response against an already-killed entry is discarded outright and never re-checked against current ROB state, since the ROB index it was issued against may already have been reused by a younger, unrelated instruction by the time a late response arrives; a discarded response writes no PRF value, sets no ready bit, wakes no reservation-station operand, completes no ROB entry, and never reaches WB0/WB1 or the writeback hold latch.

15 In-Order Commit

Execution may be out of order; architectural retirement happens strictly in order, up to two instructions at a time. The commit unit examines the two head entries together every cycle and may retire zero, one, or two, with the second slot's eligibility always conditioned on the first's: formally, `commit1` = 1 only if `commit0` succeeds as a normal, non-recovery-triggering commit, that is, head0 neither faults nor mispredicts, in addition to head1's own done/exception/write-port conditions below. Whenever head0 causes recovery or a trap, head1 does not commit that cycle regardless of its own status, since it is unconditionally squashed by head0's own recovery the same cycle (Section 16).

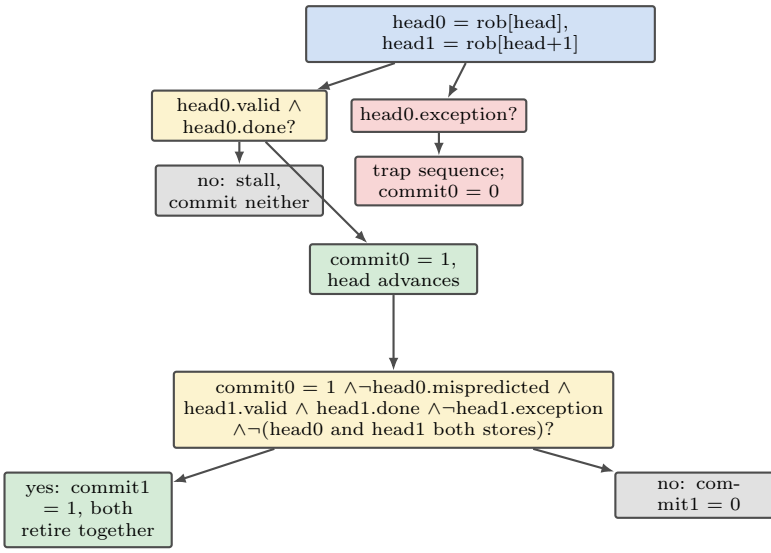

Figure 18: Two-wide commit decision tree. The second slot's path is gated entirely on the first slot's outcome.

	Condition	Result
A	both head entries done, no exception	commit both
B	head0 done, head1 not	commit head0 only
C	head0 not done	commit neither, regardless of head1
D	head0 has exception	trap; nothing commits
E	head0 commits, head1 faults	commit head0, trap on head1
F	head0 and head1 both stores	commit head0 (with its memory write); head1 waits, becomes head0 next cycle

Table 10: Two-wide commit eligibility cases.

What commit means for a register-writing instruction. There is no separate architectural register file; the physical register already holds the correct result the instant it was written back. Commit means exactly one thing: the mapping from architectural register to this physical register becomes irreversible, and the old physical register that mapping replaces becomes reclaimable

onto the free list. No data movement happens at commit. A store occupies a commit slot only in the cycle its architectural memory write can actually occur: for a single store at head0, or at head1 with head0 also committing normally, this is the same cycle it is examined; if both head0 and head1 are stores, only head0 writes memory and commits this cycle, since the single memory write port cannot serve both, and head1 is not eligible to commit until it becomes head0 the following cycle (Section 18) and can perform its own write in the same cycle it retires. An instruction is never considered retired while its required architectural side effect is still pending. For a mispredicted branch, commit triggers recovery using its `ckpt_id`, though the branch itself still commits normally that same cycle. Commit width is matched to issue width even though the two are not mathematically required to match: a machine capable of 2 executions per cycle but only 1 retirement per cycle backs the ROB up under any sustained load even with spare execution capacity.

16 Branch Recovery and Precise State Restoration

No dynamic predictor exists; fetch always assumes sequential, not-taken. Every branch or JALR is an explicitly detected candidate misprediction, resolved on EXU1 and acted on only at commit, never at resolution time, since acting early risks redirecting fetch down a path that an even older still-in-flight instruction might itself invalidate.

Question	Answer
What is saved	the full RAT and the free-list allocation pointer
When created	at dispatch, for every branch/JALR, into a free checkpoint slot; the branch's own ROB index is saved alongside
When released	the instant the branch resolves correctly, well before it commits
If none free	dispatch of a group containing a branch/JALR stalls until a slot frees

Table 11: Checkpoint lifecycle.

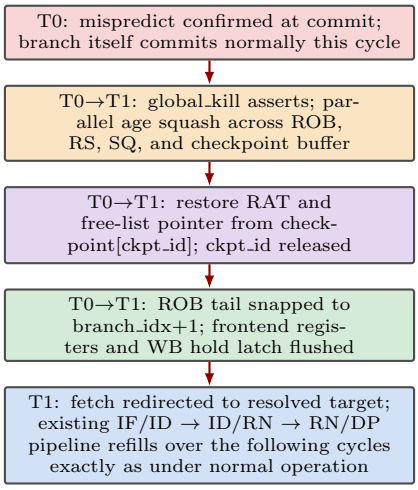

Figure 19: Checkpoint-based recovery timing. T0 is the cycle a misprediction or exception becomes authoritative at commit. The squash, checkpoint restore, and redirect all resolve within the T0-to-T1 boundary, a single cycle of speculative-state restoration; T1 is the first cycle of redirected fetch. Normal frontend refill latency (IF/ID, ID/RN, RN/DP) then applies exactly as it would for any fetch redirect, since recovery does not alter the pipeline.

Every physical register allocated by a squashed instruction is recovered implicitly by restoring the free-list allocation pointer to its checkpointed value; individual tags are never tracked or freed one by one. The one-cycle bound applies to speculative-state restoration and squash only, independent of how many instructions are discarded or how deep the ROB is, in direct contrast to a sequential walk whose cost scales with the number of in-flight instructions; useful work does not resume until the redirected fetch has refilled the frontend pipeline through dispatch, the same latency any taken redirect incurs.

Precise Exceptions

Exceptions reuse the identical parallel-squash hardware built for branch recovery. A faulting head entry never retires; every entry strictly younger than it is squashed by age comparison regardless of its own done state, since an independent younger instruction may well have finished execution long before the exception is discovered at commit. No checkpoint restore applies to an exception, since there is no speculative path to resume; fetch redirects to the trap vector instead of a branch target, and `exception_cause`, read from the faulting instruction's ROB entry, becomes available to software trap handling once the pipeline settles.

Scope. The ROB owns exactly the microarchitectural side of an exception: recording that an entry faulted, its cause code, suppressing its commit, and driving the squash and trap-vector redirect described above. The privileged CSR state, trap-vector address computation, and the software trap handler itself are treated as an external interface this core drives, not a structure this specification defines further; `exception_cause` and the redirect target are the only signals this document specifies crossing that boundary.

17 Hazard Detection and Resolution

No centralized hazard-detection unit exists. Every hazard class is resolved by whichever structure already owns the relevant information.

Hazard	Detection	Resolution	Hardware
RAW	tag not yet marked ready	value tracked by PRF ready bit and same-cycle bypass	PRF ready bits, 2-way WB bypass, 2-way RS wakeup CAM
WAR	would reuse a register a reader still needs	renaming assigns a fresh tag; old tag never reused before its own retirement	RAT + free list
WAW	two writers target the same architectural register	renaming again; last-write-wins RAT semantics	RAT, including same-cycle slot B over slot A
Structural, ports	only two ALU-class ports vs. ∼65% combined ALU/branch demand	accepted as the realistic throughput ceiling, not over-provisioned away	EXU0/EXU1 tie-break
Structural, LSU	single shared load/store port; load and store cannot issue the same cycle	store queue absorbs store timing; oldest-ready wins the port	AGU/LSU age-ordered select
Memory ordering	a load may need data from an older, unresolved store	age-ordered address check at load issue; stall on unknown, forward on match	store queue
Control	branch outcome unknown until execution	static not-taken fetch; explicit misprediction detection at commit	EXU1 branch unit, ROB
Recovery	squashing younger in-flight state after misprediction/exception	checkpointed RAT/free-list pointer restored in one cycle; parallel age squash	checkpoint buffer, ROB/RS/SQ comparators

Table 12: Hazard catalogue: detection, resolution, and owning hardware for every hazard class in the design.

17.1 Complete Hazard and Cross-Structure Interaction Matrix

Hazard handling remains fully distributed across the structures already defined in this document; no centralized hazard unit exists anywhere in this design. The six diagrams below make each interaction network visually explicit before the full closure matrix (Tables 13 through 18) enumerates every individual case with one deterministic rule apiece.

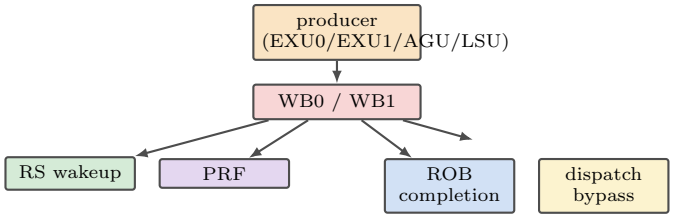

Figure 20: RAW resolution network. Every producer result reaches all four consumers through WB0/WB1; RAW is resolved by whichever consumer needs the value first.

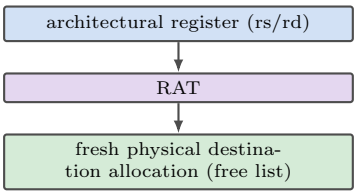

Figure 21: WAR/WAW elimination path. Every destination write allocates a new physical register through this same path, structurally preventing both hazard classes.

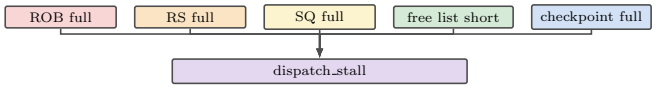

Figure 22: Structural stall network. Any one of five capacity checks, each scoped to the actual resource need of the dispatch group, can assert `dispatch_stall`.

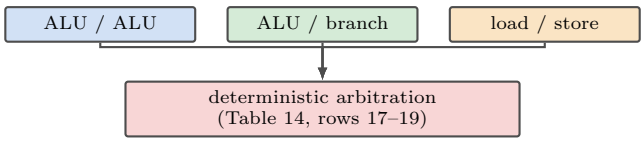

Figure 23: Execution-port conflict network. All three contention classes resolve through the same age-ordered select logic (Section 11), each with a fixed tie-break rule.

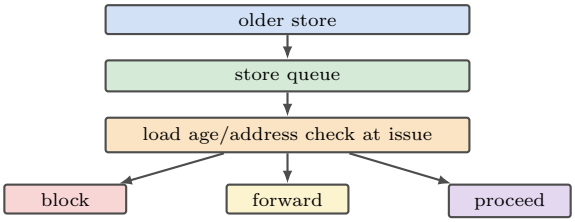

Figure 24: Memory-ordering network. Every load's issue-time check against older store-queue entries resolves to exactly one of three outcomes (Table 9).

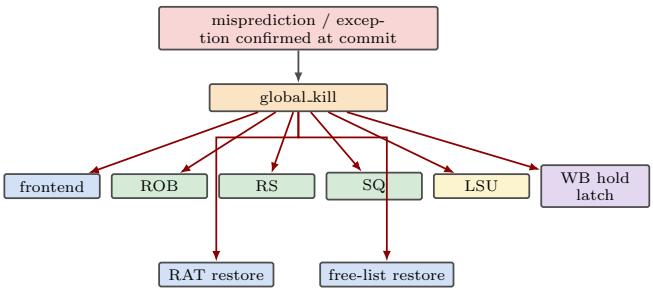

Figure 25: Recovery hazard network. A single `global_kill` assertion fans out to every structure holding speculative state; each consumer's exact response is detailed in Tables 16 and 17.

Interaction / Hazard	Detection	Resolution	Hardware	Priority / State Effect
Normal RAW dependency	consumer's source tag not yet marked ready	value tracked by PRF ready bit; consumer waits in RS until set	PRF ready bits	value available the cycle ready=1 is observed
Same-cycle WB->dispatch RAW	dispatching operand's tag equals WB0 or WB1 tag this same cycle	operand-resolution level 2 bypass supplies value/ready immediately	dispatch_unit bypass mux	bypass takes priority over the registered PRF read (level 3)
Same-group A->B RAW	slot B source register equals slot A destination register, same rename group	tag bypassed directly from slot A's freshly allocated pdst	rename same-group bypass	tag only; readiness is always forced to 0
WB0 -> RS wakeup	an RS entry's not-yet-ready source tag equals WB0 tag	that operand's ready bit set this cycle	RS wakeup comparator, port 0	evaluated independently of the WB1 comparator
WB1 -> RS wakeup	an RS entry's not-yet-ready source tag equals WB1 tag	that operand's ready bit set this cycle	RS wakeup comparator, port 1	evaluated independently of the WB0 comparator
Two WB buses waking different consumers	two distinct RS entries each match a different WB port the same cycle	both wake the same cycle with no conflict	per-entry, per-operand comparators (x2)	fully parallel; no arbitration needed between ports
x0 / has_dest-false tags must not wake	a non-destination instruction's don't-care tag could otherwise coincide with a live tag	wakeup comparators are gated on has_dest	RS entry has_dest gate	a false match is structurally impossible, not merely suppressed
WAW	two writers target the same architectural register	each writer allocates a distinct physical register at rename; RAT keeps only the last write	RAT, free list	same-cycle: slot B's write survives over slot A's
WAR	a later instruction would overwrite a register an earlier one still needs to read	renaming assigns a fresh physical tag to every writer; no register is ever reused early	RAT + free list	superseded tag lives until the writer that replaced it commits
Physical-register reclamation after commit	a committing instruction's pdst_old is known from its ROB entry	pdst_old pushed onto the free list at commit	free list push port(s)	up to 2 reclamations per cycle, matching commit width
Reuse of a physical register after recovery	tags allocated by squashed instructions are never individually freed	free-list alloc_ptr restored to its checkpointed value, implicitly un-allocating them	free list alloc_ptr restore	tags become allocatable again without per-tag tracking

Table 13: Data-dependency interaction closure (items 1 through 11).

Interaction / Hazard	Detection	Resolution	Hardware	Priority / State Effect
ROB capacity	rob_has_space_for_group false for the incoming dispatch group	dispatch_stall asserted	ROB count vs. group size	whole group stalls together, no partial allocation
RS/IQ capacity	fewer free reservation-station slots than the group needs	dispatch_stall asserted	RS occupancy count	whole group stalls together
Store queue capacity	fewer free SQ slots than needed (checked only if the group contains a store)	dispatch_stall asserted	SQ occupancy count	irrelevant, and not checked, for a store-free group
Free-list capacity	fewer free physical tags than destinations the group needs	dispatch_stall asserted	free list fl_count	checked only for slots that actually need a destination
Checkpoint buffer capacity	no free checkpoint slot (checked only if the group contains a branch/JALR)	checkpoint_full_stall asserted	checkpoint free-pool count	irrelevant, and not checked, for a branch-free group
EXU0/EXU1 ALU contention	two ALU-class candidates ready for {EXU0,EXU1} the same cycle	fixed tie-break: older age gets EXU0, next-oldest ready gets EXU1	RS select, port_mask	younger candidate remains valid, reconsidered next cycle
EXU1 branch vs. ALU contention	branch/JALR and a plain ALU op both ready for EXU1	branch/JALR wins the port	RS select priority rule	losing ALU candidate reconsidered next cycle
AGU/LSU load vs. store contention	a load and a store both ready for the single shared port	oldest-ready wins	RS select, single port_mask entry	loser remains a valid candidate, reconsidered next cycle
WB slot contention	more than 2 results demand a WB slot the same cycle	load always claims a slot; a held result claims the other; fresh EXU results fill any remainder	writeback arbiter	provably bounded to at most 1 result ever held (Section 13)
WB hold-latch occupancy	held_valid already 1 entering this cycle	wb_latch_full gates RS select for ALU/branch candidates	wb_latch_full signal	prevents oversubscribing WB before the latch drains
Single in-flight LSU port	only one AGU/LSU port exists in this design	at most one load or store issues per cycle by construction	AGU/LSU port count	identical mechanism to the load/store contention row above

Table 14: Structural-hazard interaction closure (items 12 through 22).

Interaction / Hazard	Detection	Resolution	Hardware	Priority / State Effect
Load vs. older store, unknown address	closest older SQ entry has addr_valid=0	load is not a candidate this cycle	SQ age-ordered check at load issue	load is re-evaluated every subsequent cycle
Load vs. older store, matching known address	closest older SQ entry has addr_valid=1 and its byte range is identical to the load's (addr and size both match)	forward data directly from the SQ entry, no memory access	SQ forwarding path	closest older identical-range match wins; the only forwarding case supported
Load vs. older store, non-matching address	older SQ entry's address is known and its byte range does not overlap the load's	entry ignored; age-ordered check continues to the next older entry	SQ age-ordered check, byte-range overlap test	no interaction with this load
Load vs. younger store	the store is younger than the load by ROB index	never examined; only strictly older entries are checked	SQ age comparison (rob_idx)	correct by construction of program order
Partial-overlap store/load	older store's byte range overlaps the load's byte range but the two ranges are not identical	load is correctly blocked from proceeding to memory (not treated as a non-match); no merge/partial forwarding path exists, so the load stalls until the entry retires rather than being forwarded a value	SQ byte-range overlap comparator	correctness requirement (Section 14), not a scope boundary; only the forwarding mechanism itself remains exact-range only
Store data ready after address	addr_valid=1 precedes data_valid=1 for an SQ entry	entry, and its ROB completion, wait for data_valid before forwarding or commit	SQ addr_valid/data_valid fields	ROB entry for the store is done only once both fields are valid
Store address ready after data	data_valid=1 precedes addr_valid=1 for an SQ entry	symmetric rule: entry not usable until both fields valid	SQ addr_valid/data_valid fields	same rule regardless of arrival order
SQ entry invalidation on recovery	store's ROB index is younger than the recovery point	squashed by the same parallel age comparator built for ROB/RS	SQ age comparator	entry never reaches commit; memory is never written
Store architectural side effect at commit	store reaches ROB head and commits	commit reads the SQ entry and performs the architectural memory write	commit_unit, SQ	the only point in the machine memory is ever modified
Two stores at commit, one write port	head0 and head1 are both done stores the same cycle	head0 performs its memory write and commits this cycle; head1 does not commit this cycle, since its own memory write cannot occur yet; head1 becomes head0 next cycle and commits together with its write then	commit_unit, single memory write port	an instruction is never considered retired while its required memory side effect is still pending

Table 15: Memory-hazard interaction closure (items 23 through 32).

Interaction / Hazard	Detection	Resolution	Hardware	Priority / State Effect
Conditional branch not taken	EXU1 resolves not-taken, matching the static guess	no misprediction; ROB entry marked done normally	EXU1 branch unit	commits with no recovery action
Conditional branch taken	EXU1 resolves taken, contradicting the static not-taken guess	mispredicted bit set in the ROB entry; no action yet	EXU1 branch unit, ROB	recovery deferred until the entry reaches commit
JAL	target and link value known at decode; unconditional	fetch redirected the same cycle at decode	decode_unit	never a misprediction; never enters the RS
JALR	EXU1 resolves target from a register plus immediate	target compared to the not-taken guess; mispredicted set if different	EXU1 branch unit, ROB	same commit-time recovery path as a conditional branch
Branch/JALR misprediction	mispredicted=1 in a ROB entry	full recovery sequence triggered once that entry reaches commit	ROB, branch_recovery	never acted on before commit (Section 16)
Branch outcome produced while older instructions remain in ROB	branch resolves on EXU1 before reaching the ROB head	only the ROB entry's done bit is set; no redirect yet	ROB completion	commit-time detection avoids redirecting on a path an older instruction could still invalidate
Branch reaches ROB head and mispredicts	head entry has mispredicted=1	full recovery sequence begins; branch itself still commits this cycle	commit_unit, branch_recovery	see Table 11 and Figure 19
head0 branch recovery vs. head1 retirement	head0's recovery fires this commit cycle	head1 does not retire this cycle	commit_unit, parallel squash	head1 is unconditionally squashed by head0's own age-based squash
Exception vs. head1 retirement	head0.exception=1	head1 does not retire; trap sequence begins for head0	commit_unit case D	identical precedence rule to the branch-recovery row above
Recovery vs. WB	a recovery cycle coincides with a fresh WB0/WB1 result	the producing port still completes; the targeted ROB/RS entry is written only if it survives the same-cycle squash	WB arbiter, ROB/RS squash comparators	squash and WB write are both same-cycle combinational effects; no sequencing conflict
Recovery vs. held WB result	held_valid=1 when global_kill asserts	held_valid cleared unconditionally	writeback_arbiter hold latch	exact, not approximate: see Section 13
Recovery vs. frontend progress	global_kill asserts while a fetch group is in flight	fetch PC held, then redirected; IF/ID, ID/RN, RN/DP all cleared	global_kill fan-out	the in-flight frontend group never reaches dispatch
Recovery vs. LSU in-flight state	a load is mid-flight through AGU/LSU when recovery fires	the LSU's own in-flight tracking entry for that load is killed by `global_kill` the same cycle, using its stored ROB index compared against the recovery point exactly like the ROB/RS/SQ age comparators; a late response against a killed entry is discarded outright, never re-checked against current ROB state	LSU in-flight tracking state, gated by `global_kill`	a killed transaction's response can never write PRF, wake RS, complete a ROB entry, or reach WB0/WB1, since re-checking a stale ROB index after it may have been reused by a younger instruction would be unsound
Recovery vs. SQ state	store entries younger than the recovery point	squashed by the same parallel age comparator as ROB/RS	SQ age comparator	entries never reach commit; memory is never written

Table 16: Control-hazard interaction closure (items 33 through 46).

Interaction / Hazard	Detection	Resolution	Hardware	Priority / State Effect
A->B same-cycle rename dependency	slot B's source register equals slot A's destination register	tag bypassed from slot A's fresh pdst; readiness forced to 0	rename same-group bypass	see Section 7.3 for the full worked example
A/B same-cycle WAW	both rename slots write the same architectural register	distinct physical registers allocated; only slot B's RAT write survives	RAT last-write-wins	slot A's tag becomes its own pdst_old
Checkpoint creation	a branch or JALR renames	full RAT and free-list alloc_ptr snapshot taken into a free checkpoint slot; the branch's own ROB index saved alongside	checkpoint buffer, rename_unit	exactly one snapshot per branch/JALR, at rename
Checkpoint release on correct resolution	branch/JALR resolves and is not mispredicted	checkpoint slot returned to the free pool immediately	checkpoint buffer	well before that branch itself reaches commit
Younger checkpoint invalidation after older misprediction	a checkpoint's stored ROB index is younger than the recovery point	squashed by the same parallel age comparator as ROB/RS/SQ	checkpoint buffer age comparator	returned to the free pool the same cycle (Section 7.5)
RAT restore	recovery cycle	rat[] takes checkpoint[ckpt_id].rat as a single-cycle bulk overwrite	RAT restore port	all 32 entries overwritten at once
Free-list alloc_ptr restore	recovery cycle	alloc_ptr takes checkpoint[ckpt_id].alloc_ptr	free-list alloc_ptr restore port	reclaim_ptr is unaffected
Free-list restore vs. commit reclamation	a recovery restore and a commit-time push could fall the same cycle	independent counters: restore only ever touches alloc_ptr, reclamation only ever touches reclaim_ptr	free list, two independent pointers	no interaction and no priority rule is needed
Checkpoint-full dispatch stall	no free checkpoint slot and the group contains a branch/JALR	checkpoint_full_stall asserted; whole group stalls	checkpoint buffer free-pool count	bounds the machine to 6 unresolved branches in flight

Table 17: Renaming and checkpoint interaction closure (items 47 through 55).

Interaction / Hazard	Detection	Resolution	Hardware	Priority / State Effect
ALU+ALU WB collision	EXU0 and EXU1 both produce a result the same cycle	both placed directly on WB0/WB1, no arbitration needed	writeback arbiter	at most 2 ALU-class results possible per cycle, since issue width is 2
ALU+branch WB interaction	EXU1 produces either a plain ALU result or branch metadata that cycle	same physical port, same WB path; a branch has has_dest=0 so contributes no PRF-bound value	writeback arbiter, EXU1	branch metadata still reaches the ROB via the WB path
Load+ALU WB collision	a completing load and a fresh EXU result compete for WB slots	load claims WB0 by fixed priority; the EXU result takes WB1 or the held slot	writeback arbiter priority rule	load priority, not age priority
Load+branch WB collision	a completing load and EXU1 branch resolution compete	load still claims WB0 by fixed priority; branch metadata reaches the ROB with no PRF write	writeback arbiter	a branch has no PRF-bound value, so no true PRF contention exists
Held result re-entry	a result was held in the latch last cycle	the held result claims the other WB slot this cycle, ahead of any fresh EXU result	writeback arbiter priority rule	priority order: load, then held result, then fresh EXU result
Recovery flushing held result	global_kill asserts while held_valid=1	held_valid cleared unconditionally	writeback_arbiter	see Section 13 and the control-hazard row above
WB -> PRF	WB0/WB1 valid and has_dest true	PRF write occurs at the matching physical tag	PRF write ports (2)	gated per-port by that port's own has_dest
WB -> RS	WB0/WB1 valid and has_dest true	every RS entry's matching not-yet-ready operand sets ready	RS wakeup CAM	evaluated independently per port, gated by has_dest
WB -> ROB	WB0/WB1 valid	the targeted ROB entry's done bit is set	ROB completion ports (2)	up to 2 entries marked done per cycle
WB -> dispatch bypass	WB0/WB1 tag matches a tag being dispatched the same cycle	operand-resolution level 2 bypass supplies value and readiness immediately	dispatch_unit	avoids waiting a cycle for the registered PRF read

Table 18: Writeback and completion interaction closure (items 56 through 65).

18 Same-Cycle and Corner-Case Behavior

Two instructions moving through any stage together introduces interactions a single-issue machine never faces. Table 19 gives every one this design resolves deterministically.

Scenario	Deterministic hardware rule
Both EXU0/EXU1 ready with plain ALU ops	fixed tie-break: older age assigned to EXU0, next-oldest ready to EXU1
Branch and ALU op both ready for EXU1	branch/JALR takes priority; resolving control flow early avoids extra wrong-path work
Load and store both ready for AGU/LSU	only one issues; the other remains a valid candidate and is reconsidered next cycle
Same-cycle A to B rename dependency	tag bypassed from sibling slot; readiness forced to 0 regardless
Same-cycle WAW in one rename group	slot B's RAT write survives; slot A's superseded tag reclaimed only at slot B's own commit
Writeback consumed by same-cycle dispatch	operand-resolution level 2 bypass makes it ready immediately, no wait for the registered PRF bit
Two writebacks, same cycle	structurally independent ports; cannot target the same tag by construction
Mispredict and exception on head0 and head1 the same commit cycle	head0's event is authoritative; head1 is squashed by head0's own recovery regardless of its own status
Two stores committing the same cycle	head0's store writes memory and commits this cycle; head1 does not commit this cycle even though it is done, since only one memory-write port exists; head1 becomes head0 next cycle and commits together with its own write then
Recovery and commit in the same cycle	the triggering instruction itself commits normally that cycle; it is by definition not younger than itself, so it is exempt from its own squash
Resource-full partial dispatch	a resource check evaluates only the actual number of slots that need that resource (0, 1, or 2); if either needed resource is short, the entire group stalls together, never a partial allocation
ROB allocation and commit, same cycle	disjoint index ranges by construction (allocation writes at tail, commit clears at head), so both proceed independently with no conflict
Free-list allocation and reclamation, same cycle	independent `alloc_ptr`/`reclaim_ptr` counters; both advance the same cycle with no interaction
Recovery coincides with a writeback broadcast	the writeback itself is unaffected (the producing port still completes); the hold latch is cleared unconditionally, and any ROB/RS entry the result targets is written only if that entry survives the same-cycle squash
Recovery coincides with a frontend fetch or dispatch group	`global_kill` freezes the fetch PC and clears all three frontend pipeline registers the same cycle; the group in flight through the frontend never reaches dispatch

Table 19: Deterministic resolution of every same-cycle multi-instruction interaction in the design.

19 Global Control / Stall / Flush / Redirect

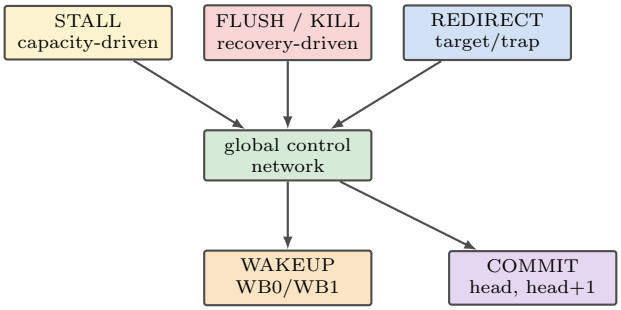

Figure 26: Global control network. Stall, flush/kill, and redirect are distinct signal classes with distinct sources; wakeup and commit are the two events that make forward progress.

Partial-group dispatch discipline. A dispatch group either allocates every resource it needs completely, or the entire group stalls together. A resource check evaluates only the actual number of instructions in the group that need that resource, zero, one, or two, never assuming both slots need it; if both slots need a destination but only one physical register is free, the whole group stalls rather than allocating to one slot alone.

20 RTL State Ownership and SystemVerilog Mapping

Each architectural block below maps to one RTL module, whose state is expressed as `typedef struct packed` records and fixed-depth arrays, sequential state in `always_ff`, and combinational logic in `always_comb`.

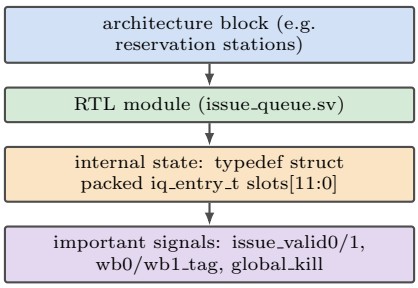

Figure 27: The architecture-to-RTL relationship followed for every block in this document: block, module, internal state, signals.

Ownership boundaries mirror Table 6: the ROB never reaches into the store queue's or checkpoint buffer's internals, and vice versa; every cross-structure interaction happens through the narrow, explicit pointer fields `sq_idx` and `ckpt_id`. Valid bits, ready bits, and head/tail/count pointers are the only sequential state most structures carry; nearly every decision (operand resolution, port selection, commit eligibility) is combina-

Signal	Driven by	Consumed by
dispatch_stall	ROB/RS/SQ/free-list/checkpoint capacity short	fetch PC hold, frontend registers freeze
global_kill	recovery active, mispredict/exception just detected	all pipeline registers, dispatch, ROB, RS, SQ, LSU, WB arbiter, commit, rename
wb_latch_full	WB hold latch occupied	RS select gate for ALU/branch candidates
sq_block	store queue age-ordered check at load issue	RS select gate for load candidates
checkpoint_full_stall	checkpoint buffer at capacity	dispatch of any group with a branch/JALR

Table 20: Global control signals.

tional logic evaluated fresh every cycle from that state.

Block	Module	Major state
Fetch	fetch_unit	fetch_pc (register)
Decode	decode_unit	none, combinational
Rename	rename_unit	rat[31:0], free-list pointers
Dispatch	dispatch_unit	none, combinational
Register file	prf	regs[59:0], ready[59:0]
Reorder buffer	rob	entries[27:0], head, tail, count
Reservation stations	issue_queue	slots[11:0]
Store queue	store_queue	entries[13:0]
Checkpoints buffer	checkpoint_ckpts[5:0]	
ALU/branch	exu0, exu1	none, combinational
Load/store	agu_lsu	in-flight load tracking
Writeback arbiter	writeback_held_result, held_valid	
Commit	commit_unit	none, combinational
Recovery recovery	branch_recovery-point latch	

Table 21: Architecture block to RTL module mapping.

State	Reset value
fetch_pc	0
Frontend registers (IF/ID, ID/RN, RN/DP)	all slots invalid
RAT	identity: rat[i] = i for all 32 entries
Free list	alloc_ptr = reclaim_ptr = 0; fl_mem holds tags 32 through 59 in order; fl_count = 28
PRF ready bits	1 for tags 0 to 31, 0 for tags 32 to 59
ROB	head = tail = count = 0; every entry invalid
Reservation stations	every slot invalid
Store queue	every entry invalid
Checkpoint buffer	all 6 slots free
WB hold latch	held_valid = 0

Table 22: Reset state of every major sequential structure.

21 Architectural Invariants

- No WAW hazard survives renaming; every destination-writing instruction gets a distinct physical register.
- No WAR hazard survives renaming; a physical register is never reused before every possible reader has had the chance to read the old mapping.
- RAW dependencies are preserved end to end through physical tags alone, never through architectural register numbers after rename.
- Instructions execute out of program order; the reservation stations select strictly on readiness, never on ROB position.
- Instructions retire strictly in program order, up to two per cycle, with the second slot always gated on the first.
- Precise exceptions are preserved: a faulting instruction's younger, even already-complete, instructions never commit.
- A mispredicted branch never leaves younger architectural state visible; squash and checkpoint restore are unconditional and complete before normal operation resumes.
- Speculative stores never modify architectural memory before commit.
- An instruction is never considered retired while its required architectural memory side effect is still pending; two same-cycle stores at head0/head1 retire one per cycle under the single write port, never both.
- A speculative result is architecturally consumable only if its producer still belongs to the surviving machine state after the recovery decision; a squashed producer's late-arriving result can never write the PRF, wake a reservation-station operand, complete a ROB entry, or occupy the writeback hold latch.
- Physical registers are never reclaimed while still needed.
- Checkpoints restore rename state (RAT and free-list pointer) in a single cycle, independent of ROB depth.
- x0 is never renamed, never allocated a physical destination, and always reads as zero.
- Two writebacks can never target the same physical register the same cycle, a structural consequence of unique physical-register allocation.
- The ROB, reservation stations, PRF, store queue, and checkpoint buffer each own exactly one kind of state; no information is duplicated across them.

22 Deliberately Omitted Features

Feature	What it does	Why omitted
Dynamic branch prediction (BTB/BHT)	reduces wrong-path fetch before misprediction is caught	recovery is already constant-time via checkpointing; not the binding throughput constraint at this scale
Second AGU/LSU port	removes load/store single-port contention	loads and stores rarely both want to issue the same cycle; added forwarding logic not justified yet
Memory dependence prediction	lets a load speculate past an uncertain older store	the conservative store-queue stall is always correct and only sometimes slower
Byte-granular partial-overlap forwarding	forwards a value even when the load's byte range only partially, not identically, overlaps an older store's range	correct blocking of a partially-overlapping load is required and already implemented (Section 14); only the merge/subset forwarding logic itself is deferred, since the conservative stall is always correct and only sometimes slower
Wider issue/fetch (3-wide+)	higher sustained throughput	would require reworking fetch through commit width together; not justified by current workload

Table 23: Features intentionally excluded from this design point, and the resulting tradeoff.

None of these are absent by oversight: each is deferred because its cost, in area, verification complexity, or both, is not repaid by the workload this core targets.

23 Final Architecture Summary

Property	This core
Width	2-wide fetch through commit
Execution	3 ports, 2 issue/cycle, OoO
Renaming	RAT + 60-entry PRF
In-flight window	28-entry ROB
Scheduling	12-entry unified reservation stations, oldest-ready select
Memory ordering	14-entry store queue, age-ordered forward/block
Branch policy	static not-taken, checkpoint recovery
Recovery latency	constant, 1-cycle squash + restore, then normal frontend refill

Table 24: Core at a glance.

This is a 2-wide, integer RV32I out-of-order core whose every structural decision, from the two-ALU-plus-shared-branch port layout to the 28-entry ROB to the 6-entry checkpoint buffer, follows directly from measured RV32I instruction mix rather than symmetric or idealized sizing. Out-of-order execution is real and load-bearing: the reservation stations select on readiness, never position, so a younger ready instruction issues ahead of an older stalled one every cycle that opportunity exists. Program order is never lost, since the ROB alone governs retirement and the only two events that can make speculative work vanish, in-order commit and checkpoint-based recovery, are both centralized, deterministic, and constant-latency regardless of how deep the machine's speculative window is at the moment they fire.

24 Future Efficiency and Scalability Upgrades

This section is descriptive only. Every item below is a **future upgrade, not part of the current design**. None of these features exist in the frozen architecture specified in Sections 1 through 23; nothing here changes a current parameter, port, signal, or diagram. The purpose is to record how this core could evolve if a specific bottleneck becomes performance-critical, and what each evolution would cost.

24.1 Dynamic Branch Prediction

Future upgrade – NOT part of the current design. Current: static not-taken, no BTB or BHT. Future: a small branch history table, most simply a per-PC 2-bit saturating counter array, consulted at fetch to guess taken/not-taken, optionally paired with a small branch target buffer for the target address. Tradeoff: less wrong-path fetch and a lower average misprediction penalty, at the cost of predictor state, fetch-stage timing and area, and training/warm-up behavior this document does not currently need to specify.

24.2 Better Branch Target Handling

Future upgrade – NOT part of the current design. A small BTB would let taken branches and JALR redirect fetch before EXU1 resolves them, rather than always fetching sequentially first. A return-address stack would specifically help JALR return-type sequences. Both become worthwhile only once static not-taken's fetch-bubble cost, not recovery latency, is the demonstrated bottleneck.

24.3 Second AGU/LSU

Future upgrade – NOT part of the current design. Current: one shared load/store port (Section 10). Future: a second independent address-generation and memory-issue port. Benefit: removes the load/store structural hazard cataloged in Table 14. Cost: a second address adder, a second store-queue access path, and doubled load-versus-store age/forwarding logic, since two loads or two stores could now be in flight against the queue the same cycle.

24.4 Larger or More Capable Store Queue

Future upgrade – NOT part of the current design. Current: 14 entries, exact-range forwarding only, with byte-range overlap already used for correct blocking (Sec-

tion 14). Future: more entries, and merge/subset forwarding logic that supplies a value even when the load's range only partially overlaps an older store's, rather than the current conservative stall on that case. This matters once a second AGU/LSU port or deeper scheduling window increases how much memory-level parallelism the machine can actually expose.

24.5 Memory Dependence Prediction

Future upgrade – NOT part of the current design. Current: a load with an older, address-unknown store simply stalls (Table 9). Future: predict that the load is independent and let it proceed speculatively, recovering if the prediction was wrong. Benefit: less unnecessary blocking. Cost: a misprediction now requires a targeted replay path this design does not currently have, since today's checkpoint recovery is scoped to control-flow speculation only.

24.6 Non-Blocking Cache / Miss Status Handling

Future upgrade – NOT part of the current design. The current specification assumes the simple existing memory model referenced throughout Sections 10 and 14. A real cache with miss status holding registers and multiple outstanding misses would meaningfully increase useful memory-level parallelism, but changes the LSU's internal timing substantially: loads would no longer complete on a fixed latency, and the writeback arbiter's bounded-demand proof (Section 13) would need to be re-derived against the new completion timing.

24.7 Wider Issue / Fetch

Future upgrade – NOT part of the current design, and not a first step. Current: 2-wide throughout (Section 3). Going 3-wide or beyond requires proportional growth in rename bandwidth, ROB allocate/commit bandwidth, RS wakeup and select width, PRF port count, writeback bandwidth, and memory bandwidth together; widening only the frontend without the rest would simply move the bottleneck downstream. This is listed last among the structural upgrades because it is the most invasive and the least likely to be the first one worth making.

24.8 Larger Scheduling Window

Future upgrade – NOT part of the current design. Current: 12-entry unified reservation stations (Section 11). A larger pool tolerates longer producer latency before issue starves, at the cost of a wider wakeup CAM and a wider age-ordered select network, both of which cost area and timing roughly in proportion to entry count.

24.9 Better Recovery / Branch Handling

Future upgrade – NOT part of the current design. Beyond dynamic prediction itself, a future design could pursue earlier (pre-commit) recovery initiation, a deeper checkpoint pool to support more outstanding speculation, or tighter frontend-redirect timing. Each trades additional complexity for a shorter effective misprediction penalty than the current commit-time, checkpoint-based mechanism.

24.10 Other Useful Future Upgrades

Future upgrade – NOT part of the current design. Upgrades that follow logically from the bottlenecks above, considered only briefly here: a realistic instruction and data cache hierarchy (a prerequisite for Section 24.6), wider writeback to match a wider backend, and a more advanced scheduler organization (for example, split ALU/memory reservation stations) if a unified pool is shown to fragment issue bandwidth at a larger size. None of these are introduced merely to lengthen this list; each is included because it is a direct consequence of one of the upgrades already discussed above.

Upgrade	Bottleneck addressed	Expected benefit	Complexity increase	Worthwhile when
1. Dynamic branch prediction / BTB	fixed static not-taken misprediction penalty	less wrong-path fetch, lower average branch penalty	predictor state, fetch-stage timing/area	branch misprediction is the demonstrated throughput bottleneck
2. Second AGU/LSU	single-port load/store structural hazard	removes load-vs-store contention	second address path, doubled forwarding logic	memory-port contention dominates measured stalls
3. Realistic cache / non-blocking memory	fixed simple memory model, no latency tolerance	real memory-level parallelism	MSHR tracking, LSU timing rework, WB re-derivation	memory latency, not port count, dominates
4. Larger scheduling window	12-entry RS starves under long producer latency	better latency tolerance	wider wakeup CAM, wider select	issue starvation is observed with existing latencies
5. Wider issue / fetch	2-wide ceiling on sustained throughput	higher peak IPC	proportional growth across rename, ROB, RS, PRF, WB, commit	only after items 2–4 no longer bound the backend
6. Memory dependence prediction	conservative load stall on unknown older store	less unnecessary blocking	replay/recovery path beyond current checkpointing	memory ordering stalls are shown to be performance-critical

Table 25: Future-upgrade priority, ranked by the bottleneck each addresses. None of these are required by, or part of, the current frozen architecture; ranking reflects likely payoff order if a bottleneck listed becomes performance-critical, not a commitment to build any of them.